

CS1010J Lecture #09 Unit 2

Basic OOP II

A paradigm shift



Department of Computer Science
School of Computing

Programming Models

PREVIOUS
LECTURE

- All the programming exercises we attempted before week 8 are **computation oriented**.
 - A solution program is composed of a single class containing a main method and some other static methods.
 - This is the style of **procedure programming**.

- Last week, we have seen another way of writing programs, in which we model entities (objects) we identify in a problem domain.
 - Each object has **data** and **behaviours** (functionalities).
 - Computation is then performed through interaction with these objects.
 - This is known as **object-oriented programming (OOP)**.

Classes and Objects

- We have studied a few standard Java classes and created objects for use in our programs.

```
Scanner sc = new Scanner(System.in);  
int data = sc.nextInt();
```

- Last week we wrote a **Circle** class to model a circle.
 - A **Circle** has one data (**radius**) and a few behaviours (e.g. to double its radius).
- We then wrote a user class (**TestCircle**) to use **Circle** class. Such a user class contains a main() method (and other static methods as appropriate).

```

class Circle {

    // attribute
    private int radius;

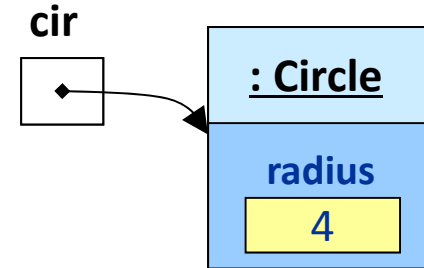
    // Constructor
    public Circle (int r) {
        radius = r;
    }

    // 2 member methods
    public void enlarge() {
        radius *= 2;
    }

    public int getRadius() {
        return radius;
    }
}

```

call constructor to
create an object



```

class TestCircle { // user class
    public static void main(String[] args) {
        Circle cir = new Circle(4);

        cir.enlarge();

        System.out.println(cir.getRadius());
    }
}

```

Invoke member methods on
an object to change its data
or retrieve its data.

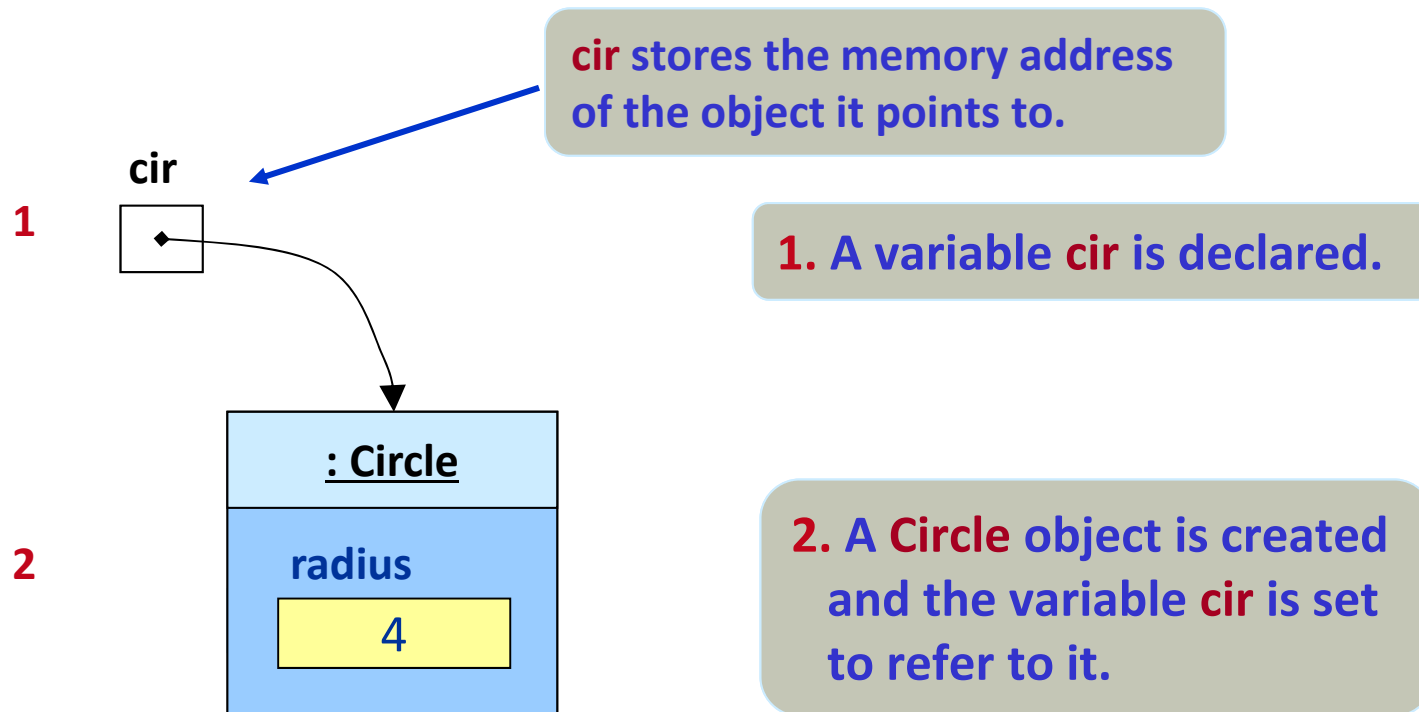
Syntax:

object.method()

Object Declaration and Creation

PREVIOUS
LECTURE

```
Circle cir = new Circle(4);
```

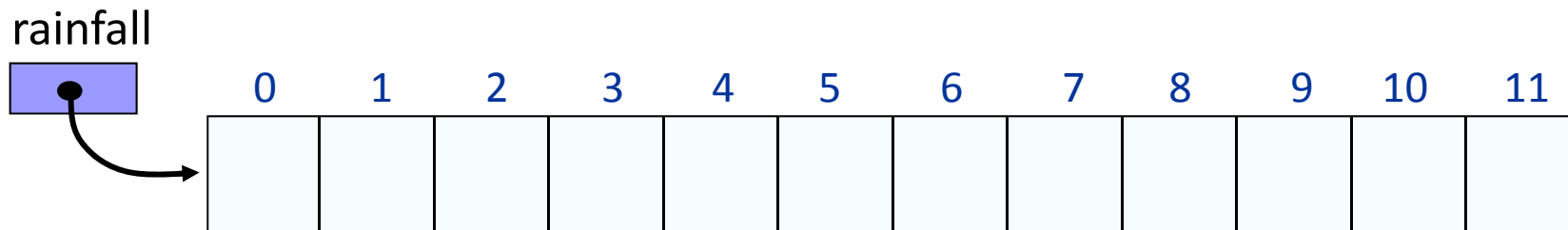


Primitive and Reference Data Types

PREVIOUS
LECTURE

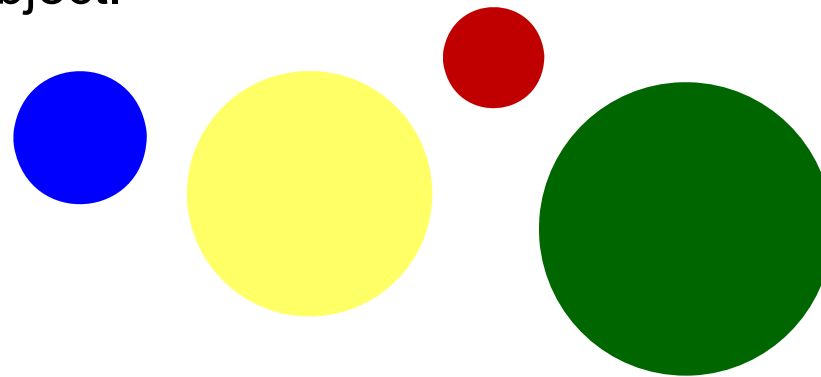
- Numerical data is called *primitive data types*.
 - `char` is also a primitive data type.
- Objects are called *reference data types*.
 - Array and String are also reference data type.
- Example:

```
double[] rainfall = new double[12];
```



Circle Class : Version 2

- Now let's add more stuffs to the **Circle** class:
 - One additional attribute **colour** to describe the colour of a circle.
 - Some additional behaviors (methods):
 - **getColour()** to return the colour of “this” **Circle** object.
 - **toString()** to return the string representation of “this” **Circle** object.
 - **equals(Circle another)** to compare “this” **Circle** object with **another** **Circle** object.



Circle Class : colour Attribute

```
class Circle {  
  
    private int radius;    // attribute 1  
    private String colour; // attribute 2  
  
    public Circle(int rad, String str) {  
        radius = rad;  
        colour = str;  
    }  
    public int getRadius() {  
        return radius;  
    }  
    public String getColour() {  
        return colour;  
    }  
    // Other member methods...  
}
```

colour is a string, a Java class we will discuss next week

```
class TestCircle { // user class  
    ...  
    Circle cir = new Circle(4, "Red");  
    System.out.println(cir.getRadius());  
    System.out.println(cir.getColour());  
    ...  
}
```

4
Red

The **toString()** Method (1/4)

- Just now we display the attributes (**colour** and **radius**) of a **Circle** object individually.
- Can we print out a **Circle** object as a whole instead?
- For example, can we achieve the following?

```
class TestCircle { // user class

    public static void main(String[] args) {

        Circle cir = new Circle(4, "Red");
        System.out.println("Circle: " + cir);
    }
}
```

Circle: [4, Red]

The **toString()** Method (2/4)

- The **toString()** method is a common method expected by every Java class.
- It is defined in the **Object** class (the root class of all Java classes) and inherited by all other classes.
- By default, the **toString()** method returns the class name and hash code of the calling object.
- A class may rewrite the **toString()** method to return the string representation of the data in an object.
 - This is called **method overriding**.

The **toString()** Method (3/4)

- In the **Circle** class, we will override the **toString()** method to return attributes (colour and radius) of a **Circle** object.

```
class Circle {  
  
    // Other code omitted for brevity  
  
    /* Override the toString() method inherited from  
     * the Object class. Return string representation  
     * of the data of the calling Circle object */  
    public String toString() {  
        return "[" + radius + ", " + colour + "];"  
    }  
}
```

The **toString()** Method (4/4)

- After the **toString()** method has been added to the **Circle** class, a user class can use it in either of these two ways:

```
// cir1 is a Circle object  
System.out.println( cir.toString() );
```

or

```
System.out.println( cir );
```

- The **toString()** method is **automatically** invoked when:
 - ❑ An object is involved in string concatenation;
 - ❑ An object is to be printed.

The **equals()** Method

- Another commonly used method is **equals()** which compares two objects to see if they contain identical data.

```
class Circle {  
    // Other code omitted for brevity  
  
    // Compare the data of two objects  
    public boolean equals(Circle another) {  
        return this.radius == another.radius &&  
               this.colour.equals(another.colour);  
    }  
}
```

Takes another **Circle** object as parameter

this means this calling object; its use is optional here

colour is a string which is also an object. Thus to compare two strings, we use the **equals()** method provided by the Java String class.

The “**this**” Keyword

- Keyword “**this**” means the calling object itself.
 - Its use is optional and implicit in most cases.

```
class Circle {  
    // Other code omitted for brevity  
    public Circle(int rad, String str) {  
        this.radius = rad; // 'this' is optional  
        this.colour = str;  
    }  
  
    public int getRadius() {  
        return this.radius;  
    }  
    ...  
}
```

```
class TestCircle { // user class  
  
    ...  
    Circle cir1 = new Circle(4, "Red");  
    System.out.println(cir1.getRadius());  
  
    Circle cir2 = new Circle(5, "Red");  
    System.out.println(cir2.getRadius());  
    ...  
}
```

Calling Methods of the Same Class

- In a class, member methods may call each other directly without the dot notation.

```
class Circle {  
  
    // Other code omitted for brevity  
    public int getRadius()    { return radius; }  
    public String getColour() { return colour; }  
  
    // Another way to write toString() method  
    public String toString() {  
        return "[" + getRadius() + ", " + getColour() + "];"  
    }  
}
```

Member methods are directly
used within the same class

Complete Program of the **Circle** Class

```
class Circle { // Version 2

    private int radius;    // attribute 1
    private String colour; // attribute 2

    // Constructor
    public Circle (int rad, String str) {
        radius = rad;
        colour = str;
    }

    // Double radius of a circle
    public void enlarge() {
        radius *= 2;
    }

    // to continue next page
```



```
// Return radius of a circle
public int getRadius() {
    return radius;
}

// Return colour of a circle
public String getColour() {
    return colour;
}

// Return string representation of the data of a circle
public String toString() {
    return "[" + radius + ", " + colour + "];"
}

// Compare two circles to see if they have identical data
public boolean equals(Circle another) {
    return this.radius == another.radius &&
           this.colour.equals(another.colour);
}
}
```

```

import java.util.*;
class TestCircleV2 { // user class version 2

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        Circle cir1 = readCircle(scanner);
        Circle cir2 = readCircle(scanner);

        System.out.println("1st circle: " + cir1);
        System.out.println("2nd circle: " + cir2);

        if ( cir1.equals(cir2) ) {
            System.out.println("Two circles have identical data");
        }
        cir1.enlarge();
        System.out.println("1st circle: " + cir1);
    }

    public static Circle readCircle(Scanner sc) { // read a circle
        System.out.print("Enter radius: ");
        int radius = sc.nextInt();
        System.out.print("Enter colour: ");
        String colour = sc.next();
        return new Circle(radius, colour); // return a Circle object
    }
}

```

```

Enter radius: 4
Enter colour: Red
Enter radius: 4
Enter colour: Red
1st circle: [4, Red]
2nd circle: [4, Red]
Two circles have
identical data
1st circle: [8, Red]

```

Today's Summary

Basic OOP II

Basic OOP concepts and features II

- **toString()** method
- **equals()** method
- The “**this**” keyword
- Comparing the contents of objects

